

SauD InoB — Programação 2

Trabalho Prático — Iteração II

Monitorização de Ocupação de Camas por Enfermaria

Grupo 5 · Programação 2 · SauD InoB · Ano letivo 2025/26

1. Introdução

Este relatório documenta as decisões de desenvolvimento tomadas na Iteração II do projeto. São descritas as funcionalidades implementadas, as opções de design adotadas e as melhorias introduzidas relativamente à Iteração I, cobrindo as principais classes do sistema: Episodio, Hospital, AnalisadorEstatistico, Visualizador, Main, GestorFicheiros, GestorMenu e RepositorioHospital

A Iteração II introduz novos requisitos funcionais — nomeadamente a visualização tabular com barras ASCII (RF3), a análise de pressão global (RF5), os gráficos de barras horizontal e vertical (RF8) e o cálculo do Índice de Pressão (RF6) — bem como requisitos não funcionais como a serialização (RF7) e a criação de testes unitários. O sistema é organizado em cinco pacotes Java: modelo (classes de domínio), utils (lógica de cálculo), main (apresentação e ponto de entrada), io (gestão de ficheiros e menus) e exceptions (exceções de domínio).

2. Classe Episodio

2.1 Descrição geral

A classe Episodio, inserida no pacote modelo, representa um internamento de um paciente numa cama específica. Na Iteração II foi adaptada para suportar serialização, requisito necessário para a persistência de dados entre sessões (RF7).

2.2 Implementação de Serializable (RF7)

A principal alteração introduzida na Iteração II foi a implementação da interface Serializable, acompanhada da declaração explícita do serialVersionUID:

Elemento	Valor / Tipo	Justificação
implements Serializable	Interface java.io.Serializable	Permite que objetos Episodio sejam gravados e lidos de ficheiros .ser
serialVersionUID	1L	Garante compatibilidade entre versões serializadas; evita InvalidClassException ao ler ficheiros gerados numa versão anterior

```
import java.io.Serializable;

/**
 * Representa um episódio de internamento.
 */
public class Episodio implements Serializable { 74 usages  T1ag013 +1
    private static final long serialVersionUID = 1L; no usages
```

A adição do serialVersionUID é considerada boa prática mesmo quando o valor é 1L, pois torna explícita a intenção de compatibilidade e evita que a JVM gere um valor automático (que pode variar entre compilações e causar erros de desserialização).

2.3 Atributos e lógica principal

A lógica da classe manteve-se idêntica à Iteração I. Os atributos principais são:

Atributo	Tipo	Descrição
episodioid	String	Identificador único gerado automaticamente com prefixo EP e contador estático
camald	String	Identificador da cama onde ocorre o internamento
dataAdmissao	LocalDate	Data de entrada do paciente
dataAlta	LocalDate	Data de saída; null se o episódio ainda estiver ativo
estado	String	ATIVO ou ALTA; atualizado automaticamente ao dar alta
enfermariald	String	Associado após criação, quando o episódio é adicionado a uma enfermaria

2.4 Consideração sobre o contador estático

O atributo proximoNumero é declarado como static, o que implica que o seu valor não é preservado durante a serialização de objetos individuais. Em sessões que misturem objetos desserializados de ficheiro com novos objetos criados em runtime, existe risco de colisão de identificadores. Esta limitação é conhecida e aceitável no contexto do projeto, uma vez que o identificador serve apenas como referência visual e não como chave primária de base de dados.

```
/** Contador para gerar identificadores simples. */
private static int proximoNumero = 1; 1 usage
```

2.5 Métodos principais

Método	Descrição
estaAtivoEm(LocalDate)	Verifica se o episódio estava ativo numa data: após a admissão e antes ou na data de alta (se existir)
darAlta(LocalDate)	Regista a alta se a data não for anterior à admissão; atualiza o estado para ALTA
calcularLoS()	Calcula o Length of Stay em dias usando ChronoUnit.DAYS; devolve -1 se não houver alta
temAlta()	Devolve true se a data de alta não for null

associarEnfermaria(String)

Define o enfermariald após o episódio ser adicionado a uma enfermaria

```
public boolean estaAtivoEm(LocalDate data) { 4 usages  ⓘ T1ag013
    if (data == null || dataAdmissao == null) {
        return false;
    }
    boolean depoisDaAdmissao = !data.isBefore(dataAdmissao);
    boolean antesOuNaAlta = dataAlta == null || !data.isAfter(dataAlta);
    return depoisDaAdmissao && antesOuNaAlta;
}
```

```
public long calcularLoS() { 3 usages  ⓘ T1ag013
    if (!temAlta()) {
        return -1;
    }
    return ChronoUnit.DAYS.between(dataAdmissao, dataAlta);
}
```

3. Classe Hospital

3.1 Descrição geral

Esta classe atua como o ponto central de agregação do sistema, sendo reformulada nesta iteração para centralizar a gestão global das enfermarias e delegar os cálculos matemáticos complexos. Além de encapsular os atributos fundamentais e garantir a robustez na manipulação das enfermarias (prevenindo duplicados e valores nulos), a classe foi integrada com o AnalisadorEstatistico através do método `listarEnfermariasOrdenadasPorTaxaOcupacao(LocalDate)`. Esta alteração liberta o Hospital da responsabilidade direta de computação algorítmica, cumprindo o princípio da responsabilidade única (SRP) e focando-se estritamente na coordenação do modelo de dados.

```
public boolean adicionarEnfermaria(Enfermaria enfermaria) { 14 usages  T1ag013
    if (enfermaria == null || obterEnfermaria(enfermaria.getIdentificador()) != null) {
        return false;
    }
    enfermarias.add(enfermaria);
    return true;
}

public Enfermaria obterEnfermaria(String identificador) { 9 usages  T1ag013
    for (Enfermaria enfermaria : enfermarias) {
        if (enfermaria.getIdentificador().equalsIgnoreCase(identificador)) {
            return enfermaria;
        }
    }
    return null;
}

public List<Enfermaria> getEnfermarias() { return new ArrayList<>(enfermarias); }

public List<Enfermaria> listarEnfermariasOrdenadasPorTaxaOcupacao(LocalDate data) { 3 usages  T1ag013 +1
    return AnalisadorEstatistico.ordenarPorTaxaOcupacao(this.enfermarias, data);
}

@Override  T1ag013
public String toString() { return String.format("%s | Enfermarias: %d", nome, enfermarias.size()); }
```

3.2 Estratégia de arranque e inicialização

No início da aplicação, a classe Main chama `GestorMenu.configurarArranque`, que apresenta ao utilizador três opções para definir a origem dos dados:

Opção	Comportamento
1 — Objetos predefinidos	Cria automaticamente um conjunto de enfermarias e episódios hardcoded, útil para demonstração e testes imediatos das funcionalidades.
2 — Hospital vazio	Inicia a aplicação sem dados, permitindo ao utilizador inserir enfermarias e episódios manualmente através do menu.
3 — Ficheiro binário (.dat)	Carrega um estado previamente gravado por serialização, restaurando todas as enfermarias e episódios da sessão anterior (RF7).

O método devolve o objeto Hospital já inicializado ao Main, que entra de seguida num ciclo do-while com o menu principal.

4. Classe AnalisadorEstatistico

4.1 Descrição geral

A classe AnalisadorEstatistico, inserida no pacote utils, é responsável por todos os cálculos estatísticos e indicadores hospitalares do sistema. Na Iteração II foi significativamente expandida para suportar os novos requisitos funcionais, mantendo estritamente o Princípio da Responsabilidade Única (SRP): a classe não efetua qualquer impressão na consola, tratando exclusivamente do processamento de dados matemáticos.

4.2 Evolução face à Iteração I

Na Iteração I, a classe continha apenas o cálculo de estatísticas de LoS e a ordenação por taxa de ocupação. Na Iteração II foram adicionados métodos para suportar os requisitos RF3, RF5 e RF6, bem como um conjunto de constantes nomeadas que centralizam os limiares de pressão e as escalas de classificação:

Versão	Métodos principais	Pacote
Iteração I	calcularEstatisticaLoS, ordenarPorTaxaOcupacao, analisarPressaoPorIntervalo	utils
Iteração II	calcularTurnover, percentagemEmPressao, contarEmPressao, calcularIndiceDePressao, ordenarPorIndiceDePressao, calcularScoreOcupacao, calcularScoreTurnover, interpretarIndice	utils

4.3 Constantes de limiar

Uma das principais decisões de design foi a centralização de todos os limiares de classificação em constantes privadas (e uma pública). Esta abordagem garante que qualquer alteração futura aos critérios clínicos exige modificação num único local:

Constante	Valor	Utilização
LIMIAR_PRESSAO (public)	85.0	Determina se uma enfermaria está em pressão; partilhada com o Visualizador
LIMIAR_OCUPACAO_NIVEL_2/3/4	90.0 / 95.0 / 100.0	Escalas do score de ocupação para o Índice de Pressão (RF6)
LIMIAR_TURNOVER_NIVEL_1/2/3/4	10.0 / 20.0 / 30.0 / 40.0	Escalas do score de turnover para o Índice de Pressão (RF6)
LIMIAR_INDICE_BAIXA / MODERADA	2.0 / 3.5	Interpretação textual do Índice de Pressão calculado

```
// Constantes de Ocupação
/** Limiar percentual a partir do qual uma enfermaria é considerada em pressão. */
public static final double LIMIAR_PRESSAO = 85.0; 2 usages
private static final double LIMIAR_OCUPACAO_NIVEL_2 = 90.0; 1 usage
private static final double LIMIAR_OCUPACAO_NIVEL_3 = 95.0; 1 usage
private static final double LIMIAR_OCUPACAO_NIVEL_4 = 100.0; 1 usage

private static final double LIMIAR_TURNOVER_NIVEL_1 = 10.0; 1 usage
private static final double LIMIAR_TURNOVER_NIVEL_2 = 20.0; 1 usage
private static final double LIMIAR_TURNOVER_NIVEL_3 = 30.0; 1 usage
private static final double LIMIAR_TURNOVER_NIVEL_4 = 40.0; 1 usage

private static final double LIMIAR_INDICE_BAIXA = 2.0; 1 usage
private static final double LIMIAR_INDICE_MODERADA = 3.5; 1 usage
```

4.4 Cálculo do Índice de Pressão (RF6)

O Índice de Pressão é calculado através de uma pipeline de três métodos encadeados, o que permite reutilizar cada componente de forma independente em testes unitários:

Método	Responsabilidade
calcularScoreOcupacao(percOcup)	Converte a taxa de ocupação num score de 1 a 5 conforme as faixas do enunciado
calcularScoreTurnover(percTurnover)	Converte a percentagem de turnover num score de 1 a 5
calcularIndiceDePressao(enf, data)	Pondera os dois scores (70% ocupação + 30% turnover) e arredonda a 1 decimal
interpretarIndice(indice)	Devolve a classificação textual: Pressão Baixa / Moderada / Alta
ordenarPorIndiceDePressao(lista, data)	Gera uma cópia ordenada de forma decrescente pelo índice calculado

```
public static double calcularIndiceDePressao(Enfermaria enf, LocalDate data) { 3 usages 2 Tag013 +1
    double percOcup = enf.getTaxaOcupacao(data);
    double percTurnover = calcularTurnover(enf, data);

    int scoreOcup = calcularScoreOcupacao(percOcup);
    int scoreTurnover = calcularScoreTurnover(percTurnover);

    return Math.round((0.7 * scoreOcup + 0.3 * scoreTurnover) * 10.0) / 10.0;
}
```

4.5 Métodos de pressão global (RF5)

Os métodos `percentagemEmPressao` e `contarEmPressao` foram implementados como métodos de classe (static), conforme exigido pelo enunciado. O segundo serve de auxiliar ao primeiro, evitando duplicação de lógica, e é também utilizado diretamente pelo Visualizador para apresentar o detalhe numérico na consola. A decisão de separar a contagem da percentagem permite que o Visualizador mostre simultaneamente o valor absoluto ("3 de 5") e a percentagem ("60.0%") sem recalculá-lo o mesmo ciclo duas vezes.

```
public static double percentagemEmpressao(List<Enfermaria> enfermarias, LocalDate data) {
    if (enfermarias == null || enfermarias.isEmpty()) {
        return 0.0;
    }
    int emPressao = contarEmpressao(enfermarias, data);
    return (emPressao * 100.0) / enfermarias.size();
}
```

4.6 Correção do nome do método de LoS

Na Iteração I o método continha um erro tipográfico: `calculusEstatisticaLoS` (com 's' a mais). Na Iteração II foi corrigido para `calcularEstatisticaLoS`, alinhando a nomenclatura com a convenção camelCase do restante código. O método `ordenarPorTaxaOcupacao` foi também refatorado para devolver uma nova lista em vez de modificar a lista original, tornando o comportamento mais seguro e previsível.

5. Classe Visualizador

5.1 Descrição geral

A classe `Visualizador`, inserida no pacote `main`, é responsável por toda a apresentação visual de dados na consola. Foi criada de raiz na Iteração II para centralizar a camada de apresentação, em complemento ao `AnalizadorEstatistico` que trata exclusivamente da lógica de cálculo. Todos os métodos são estáticos e a classe não deve ser instanciada.

5.2 Separação de responsabilidades (SRP)

Na Iteração I, o método `analisarPressaoPorIntervalo` existia no `AnalizadorEstatistico` e combinava cálculo com impressão na consola, violando o SRP. Na Iteração II, esse método foi movido para o `Visualizador` (`mostrarAnalisePressaoIntervalo`), e o `AnalizadorEstatistico` passou a ser uma classe puramente de cálculo. Esta separação facilita também a escrita de testes unitários, pois o `AnalizadorEstatistico` pode ser testado sem depender de saída na consola.

5.3 Constantes e método auxiliar `construirBarraHorizontal`

A barra ASCII é construída pelo método privado `construirBarraHorizontal`, que recebe uma percentagem e um símbolo configurável. A escala é de 50 caracteres para 100%, conforme especificado no enunciado. O método limita a percentagem a 100.0 antes de calcular o comprimento, garantindo que barras nunca extravasam o espaço disponível mesmo em situações de sobrelotação.

Constante	Valor	Descrição
TAMANHO_BARRA_ASCII	50	Número de caracteres que representa 100% na barra horizontal
ALTURA_GRAFICO_VERTICAL	20	Número máximo de linhas do gráfico vertical

```
private static final int ALTURA_GRAFICO_VERTICAL = 20; 4 usages
private static final int TAMANHO_BARRA_ASCII = 50; 2 usages
```


5.4 Tabela de ocupação (RF3)

A funcionalidade RF3 é implementada por dois métodos complementares:

Método	Âmbito	Dados apresentados
mostrarTabelaOcupacao	Uma enfermaria, intervalo de datas	Uma linha por dia: ocupadas, camas totais, %Ocup, Turnover%, barra ASCII
mostrarTabelaOcupacaoMultipla	Várias enfermarias, uma data	Uma linha por enfermaria: mesmas colunas da tabela simples

Ambos os métodos delegam no `AnalizadorEstatistico.calcularTurnover` para obter a percentagem de turnover diário, mantendo a separação entre cálculo e apresentação. A validação de parâmetros nulos e de intervalo inválido é feita no início de cada método, com mensagem de erro clara e retorno imediato.

5.5 Gráficos de barras (RF8)

São implementados dois tipos de gráfico, ambos com símbolo configurável pelo utilizador:

Método	Orientação	Escala
mostrarGraficoHorizontal	Horizontal	A barra mais longa ocupa 50 caracteres (100%); restantes são proporcionais ao valor passado
mostrarGraficoVertical	Vertical	Altura máxima de 20 linhas; alturas calculadas proporcionalmente ao valor máximo da lista; arredondamento com <code>Math.round</code> para maior fidelidade visual

No gráfico vertical, a altura de cada barra é calculada relativamente ao valor máximo da lista (e não a um máximo fixo de 100), o que garante que a barra mais alta ocupa sempre toda a altura disponível independentemente da escala dos dados.

5.6 Ranking do Índice de Pressão (RF6)

O método `mostrarRankingIndicePressao` delega completamente o cálculo no `AnalizadorEstatistico` (`ordenarPorIndiceDePressao`, `calcularIndiceDePressao`, `interpretarIndice`) e limita-se a formatar e apresentar a tabela de ranking. A tabela inclui posição, identificador, %Ocup, %Turnover, score de ocupação, score de turnover, índice final e classificação textual.

6. Classe Main

6.1 Descrição geral

Ponto de entrada da aplicação. Chama `GestorMenu.configurarArranque` para inicializar o hospital e percorre um ciclo do-while que apresenta o menu e despacha cada opção para o método correspondente do `GestorMenu`. O uso do do-while elimina a necessidade de uma variável inicializada artificialmente para entrar no ciclo.

6.2 Ciclo principal

```
int opcao;
do {
    printMenu(hospital.getNome());
    opcao = GestorMenu.lerInteiro(leitor, mensagem: "Opcao: ", min: 0, OPCAO_MAXIMA);

    try {
        switch (opcao) {
            case 1 -> GestorMenu.carregarCSV(leitor, hospital);
            case 2 -> GestorMenu.inserirEnfermaria(leitor, hospital);
            case 3 -> GestorMenu.inserirEpisodio(leitor, hospital);
            case 4 -> GestorMenu.mostrarGraficosOcupacao(leitor, hospital);
            case 5 -> GestorMenu.analisarPressaoIntervalo(leitor, hospital);
            case 6 -> GestorMenu.mostrarListagensOrdenadas(leitor, hospital);
            case 7 -> GestorMenu.alterarCapacidadeEnfermarias(leitor, hospital);
            case 8 -> GestorMenu.mostrarEnfermariasEmPressao(leitor, hospital);
            case 9 -> GestorMenu.mostrarRankingIndicePressao(leitor, hospital);
            case 10 -> GestorMenu.gravarEstado(leitor, hospital);
            case 11 -> hospital = GestorMenu.carregarEstado(leitor);
            case 0 -> System.out.println("\nAte Logo!");
        }
    } catch (Exception e) {
        System.out.println("[ERRO] " + e.getMessage());
    }
} while (opcao != 0);
```

O método `configurarArranque` devolve o objeto `Hospital` já inicializado ao `Main`, que o mantém como variável local durante toda a execução da aplicação.

7. Classe GestorFicheiros

7.1 Descrição geral

Responsável pela leitura e validação de ficheiros CSV de enfermarias e episódios. Adotou-se uma estratégia de validação em duas camadas para separar claramente a verificação da criação de objetos.

7.2 Validação em duas camadas

O método `validarLinhaEnfermaria` verifica os campos base (tipo, ID, capacidade) e delega nos métodos `validarCamposGeral`, `validarCamposPsiquiatria` e `validarCamposCuidadosIntensivos`. Se a validação falhar, a linha é ignorada e o erro registado no log. Os métodos `processarEnfermaria...` apenas criam os objetos após validação aprovada.

```

while (sc.hasNextLine()) {
    linha++;
    String[] d = sc.nextLine().trim().split(regex: ".");

    // A barreira de entrada. Se a linha não for válida, salta para a próxima imediatamente.
    if (!validarLinhaEnfermaria(d, linha)) continue;

    String tipo = d[0].trim().toUpperCase();
    String id    = d[1].trim();
    int cap      = Integer.parseInt(d[2].trim());

    if (tipo.equals("GERAL")) {
        processarEnfermariaGeral(d, id, cap, h);
    } else if (tipo.equals("PSIQUIATRICA")) {
        processarEnfermariaPsiquiatrica(d, id, cap, h);
    } else if (tipo.equals("INTENSIVOS")) {
        processarEnfermariaCuidadosIntensivos(d, id, cap, h);
    }
}
sc.close();

```

7.3 Log de erros

Todos os erros de validação são registados no ficheiro `erros_validacao.log` com a data e número de linha, usando `try-with-resources` para garantir que o ficheiro é sempre fechado, mesmo em caso de exceção durante a escrita.

8. Classe GestorMenu

8.1 Descrição geral

Centraliza toda a interação com o utilizador. Contém métodos estáticos para cada opção do menu e métodos de leitura validada de input. O tratamento de exceções garante que entradas inválidas não interrompem o programa.

8.2 Leitura de input validado

Os métodos lerInteiro, lerDecimal e lerData utilizam um ciclo while(!entradaValida) com try-catch para capturar erros de formato, repetindo o pedido até obter uma entrada válida.

```
public static int lerInteiro(Scanner leitor, String mensagem, int min, int max) { 6 usages 2 Tiag
    int valor = 0;
    boolean entradaValida = false;

    while (!entradaValida) {
        System.out.print(mensagem);
        String input = leitor.nextLine();

        try {
            valor = Integer.parseInt(input);

            if (valor >= min && valor <= max) {
                entradaValida = true;
            } else {
                System.out.println("[ERRO] Valor deve estar entre " + min + " e " + max + ".");
            }
        } catch (NumberFormatException e) {
            System.out.println("[ERRO] Entrada invalida. Introduza um numero inteiro.");
        }
    }
    return valor;
}
```

8.3 RF4 — Alteração de Capacidade (static)

O método alterarCapacidadeEnfermarias lê a percentagem de variação e delega no método static Enfermaria.alterarCapacidade(lista, percentagem), que percorre todas as enfermarias, calcula as novas camas e garante o mínimo de 1 cama.

```

public static void alterarCapacidade(List<Enfermaria> enfermarias, double percentagem) { 3 usages
    if (enfermarias == null || enfermarias.isEmpty()) {
        return;
    }

    for (int i = 0; i < enfermarias.size(); i++) {
        Enfermaria enf = enfermarias.get(i);
        int capacidadeAtual = enf.getNumeroCamas();

        // Calcula a diferença e arredonda para não termos "meias camas"
        int variacao = (int) Math.round(capacidadeAtual * (percentagem / 100.0));
        int novaCapacidade = capacidadeAtual + variacao;

        // Blindagem: Uma enfermaria tem de ter, no mínimo, 1 cama.
        if (novaCapacidade < 1) {
            novaCapacidade = 1;
        }

        // Como o método está dentro da própria classe, pode aceder ao atributo privado diretamente
        enf.numeroCamas = novaCapacidade;
    }
}

```

8.4 RF5 — Percentagem de Enfermarias em Pressão (static)

O método `mostrarEnfermariasEmPressao` atua apenas como controlador: solicita a data de referência ao utilizador e delega toda a lógica de contagem, cálculo percentual e apresentação gráfica na classe `Visualizador` através do método `mostrarPercentagemEmPressao`. Isto garante a separação de responsabilidades.

```

public static void mostrarEnfermariasEmPressao(Scanner leitor, Hospital hospital) throws HospitalException {
    System.out.println("\n--- Percentagem de Enfermarias em Pressao ---");
    validarHospitalNaoVazio(hospital);

    LocalDate data = lerData(leitor, mensagem: "Data de referencia (" + FORMATO_DATA_ESPERADO + "): ");

    main.Visualizador.mostrarPercentagemEmPressao(hospital.getEnfermarias(), data);
}

```

8.5 RF6 — Ranking do Índice de Pressão

O método `mostrarRankingIndicePressao` lê a data solicitada e delega a exibição do ranking diretamente na classe `Visualizador`. Os cálculos matemáticos do Índice de Pressão e a ordenação decrescente da lista foram abstraídos e são tratados internamente pela interação entre o `Visualizador` e o `AnalizadorEstatistico`, libertando o `GestorMenu` de cálculos complexos e algoritmos de ordenação.

```

public static void mostrarRankingIndicePressao(Scanner leitor, Hospital hospital) throws HospitalException {
    System.out.println("\n--- Ranking de Índice de Pressao ---");
    validarHospitalNaoVazio(hospital);

    LocalDate data = lerData(leitor, mensagem: "Data de referencia (" +
        "" + FORMATO_DATA_ESPERADO + "): ");

    main.Visualizador.mostrarRankingIndicePressao(hospital.getEnfermarias(), data);
}

```

9. Classe RepositórioHospital

9.1 Descrição geral

Esta classe é responsável pela persistência de dados do sistema, tendo como única função gravar e carregar o estado global do objeto Hospital em disco. Para isso, utiliza a serialização nativa do Java (ObjectOutputStream e ObjectInputStream), convertendo o grafo completo de objetos (hospital, enfermarias e episódios) num fluxo de bytes binários armazenados num ficheiro, garantindo a recuperação integral do estado da aplicação entre execuções.

```
/**
 * Converte o objeto Hospital em bytes e guarda-o num ficheiro.
 *
 * @param hospital O hospital a gravar.
 * @param ficheiro O nome do ficheiro destino.
 * @throws IOException Se houver um erro de escrita no disco.
 */
public static void gravarEstado(Hospital hospital, String ficheiro) throws IOException {
    try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(ficheiro))) {
        oos.writeObject(hospital);
    }
}

/**
 * Le um ficheiro de bytes e reconstrói o objeto Hospital.
 *
 * @param ficheiro O nome do ficheiro a ler.
 * @return O objeto Hospital reconstruído.
 * @throws IOException Se o ficheiro não existir ou não puder ser lido.
 * @throws HospitalException Se os dados dentro do ficheiro não corresponderem a um Hospital.
 */
public static Hospital carregarEstado(String ficheiro) throws IOException, HospitalException {
    try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(ficheiro))) {
        return (Hospital) ois.readObject();
    } catch (ClassNotFoundException e) {
        throw new HospitalException("O ficheiro não contém um estado válido do Hospital.");
    }
}
```

10. Testes Unitários

Foram desenvolvidos 79 testes unitários distribuídos por 6 classes de teste, cobrindo as principais funcionalidades do sistema. Os testes foram escritos com JUnit e seguem o padrão de organização por classe testada.

10.1 EpisodioTest — 16 testes

Testa a classe central do sistema. Cobre temAlta(), calcularLoS(), darAlta() com datas válidas e inválidas, estaAtivoEm() em vários cenários (antes da admissão, durante o internamento, no dia da alta e depois da alta), getEstado(), getCamald(), getDataAdmissao() e associarEnfermaria(). Inclui casos limite como LoS zero e alta na mesma data da admissão.

10.2 EnfermariaTest — 16 testes

Testa a lógica de negócio principal da enfermaria. Cobre adicionarEpisodio() com episódio válido e nulo, getOcupacaoAbsoluta() dentro e fora do intervalo, getTaxaOcupacao(), emPressao() nos dois estados, getNumeroAdmissoes(), getNumeroAltas(), getValoresLoS(), alterarCapacidade() com percentagem positiva e com blindagem mínima, getEpisodiosOrdenadosPorAdmissao() e existeConflitoDeCama() com e sem conflito e com cama diferente.

10.3 EnfermariaGeralTest — 8 testes

Testa os métodos específicos da enfermaria geral. Cobre adicionarRecurso() com valor válido, nulo e em branco, removerRecurso() com recurso existente e inexistente, getRecursosDisponiveis() verificando a cópia defensiva, getLimiteAcompanhantes() e getHorariosVisitas().

10.4 HospitalTest — 12 testes

Testa a gestão do hospital. Cobre getNome(), adicionarEnfermaria() com enfermaria válida, nula e duplicada, obterEnfermaria() com identificador existente, inexistente e insensível a maiúsculas, getEnfermarias(), listarEnfermariasOrdenadasPorTaxaOcupacao() verificando a primeira e última posição, toString().

10.5 AnalisadorEstatisticoTest — 22 testes

Testa os cálculos estatísticos e indicadores. Cobre calcularScoreOcupacao() nos 5 intervalos incluindo os limites exatos, calcularScoreTurnover() nos 5 intervalos, interpretarIndice() nas 3 classificações, calcularEstatisticaLoS() verificando total de episódios, média, mínimo, máximo e caso sem altas, percentagemEmPressao() com lista em pressão, sem pressão e vazia.

10.6 RepositorioHospitalTest — 5 testes

Testa a persistência de dados. Cobre gravação e carregamento do nome do hospital, das enfermarias e dos episódios, verificação da existência do ficheiro após gravação e lançamento de exceção ao carregar ficheiro inexistente. O ficheiro de teste é apagado automaticamente no @AfterAll.

Classe de Teste	Testes	Principais funcionalidades cobertas
EpisodioTest	16	temAlta, calcularLoS, darAlta, estaAtivoEm, associarEnfermaria
EnfermariaTest	16	adicionarEpisodio, ocupação, pressão, admissões, altas, LoS, conflitos de cama
EnfermariaGeralTest	8	adicionarRecurso, removerRecurso, cópia defensiva, limites e horários
HospitalTest	12	adicionarEnfermaria, obterEnfermaria, listar, ordenar por taxa de ocupação
AnalisadorEstatisticoTest	22	scores (5 intervalos), interpretarIndice, calcularLoS, percentagemEmPressao
RepositorioHospitalTest	5	gravar estado, carregar estado, ficheiro inexistente
Total	79	—

11. Conclusão

As classes documentadas neste relatório cobrem os requisitos funcionais RF3, RF5, RF6 e RF7 da Iteração II, bem como a base necessária para os testes unitários.

As principais decisões de design foram: aplicar o SRP de forma rigorosa, separando completamente o cálculo (AnalisadorEstatistico) da apresentação (Visualizador); centralizar todos os limiares em constantes nomeadas para facilitar manutenção futura; implementar o Índice de Pressão como uma pipeline de métodos independentes e testáveis individualmente; adaptar a classe Episodio para suportar serialização com o mínimo de alterações à lógica existente; e delegar na classe Hospital a coordenação central do modelo de dados, integrando-a com o AnalisadorEstatistico para manter o SRP.